ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos

StickMania – Jogo de Hóquei em Patins 2D

Rafael da Hora Silveira

junho de 2027



ESCOLA SECUNDÁRIA DA BOA NOVA

Curso Profissional de Técnico/a de Desenvolvimento de Software

StickMania – Jogo de Hóquei em Patins 2D

Sob o acompanhamento do Dr. Fernando Filipe

Rafael da Hora Silveira e Vasco Andreia

junho de 2027

Agradecimentos

Agradecemos ao nosso orientador, Dr. Fernando Filipe, e à professora Cristina Paes pelo acompanhamento, pelas sugestões e pela disponibilidade ao longo do projeto. Agradecemos também aos professores do curso, aos colegas que testaram o jogo e deram opiniões, e às nossas famílias pelo apoio.

Resumo

Este relatório descreve o StickMania, um jogo de computador 2D de hóquei em patins desenvolvido em Python com a biblioteca pygame-ce, no âmbito da Prova de Aptidão Profissional. O jogador controla uma equipa contra o computador, com as oito equipas portuguesas, os seus símbolos e equipamentos. O jogo tem física da bola com atrito e ressaltos, colisões entre jogadores, faltas, guarda-redes, uma inteligência artificial com papéis táticos e três níveis de dificuldade, menus completos e uma câmara em perspetiva que mostra o rinque como numa transmissão de televisão. O desenvolvimento foi iterativo, com versões jogáveis sucessivas e controlo de versões no GitHub. O relatório apresenta a análise de requisitos, a modelação (casos de uso, arquitetura, classes, estados, fluxogramas e modelo de dados), a implementação dos algoritmos principais, os testes realizados e a gestão do projeto. Como trabalho futuro está previsto um modo torneio com base de dados SQLite.

Índice

1. Introdução 10
1.1 Enquadramento e contextualização do tema 10
1.2 Motivação e pertinência do projeto 10
1.3 Objetivos (geral e específicos) 10
1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS 11
1.5 Estrutura do relatório 11
2. Análise do sistema 12
2.1 Descrição do problema e da necessidade a resolver ◆ MADS 12
2.2 Intervenientes e contexto de utilização ◆ MADS 12
2.3 Levantamento e análise de requisitos ◆ MADS 13
2.3.1 Técnicas de levantamento utilizadas ◆ MADS 13
2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS 13
2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS 14
2.4 Modelação do sistema ◆ MADS 14
2.4.1 Cenários de utilização / casos de uso ◆ MADS 14
2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS 15
2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS 17
2.5 Projeto do hardware 19
2.5.1 Componentes: sensores, atuadores e controlador 19
2.5.2 Esquema elétrico e ligações 19
2.5.3 Estrutura mecânica 20
2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS 20
3. Implementação 20
3.1 Montagem do hardware 20
3.2 Programação do controlo (lógica e algoritmos) 21
3.3 Integração hardware – software 23
3.4 Decisões técnicas e dificuldades ultrapassadas 23
4. Testes e validação 24
4.1 Plano de testes 24
4.2 Ensaios, calibração e medições 25
4.3 Resultados obtidos e ajustes 26
5. Gestão do projeto 26
5.1 Planeamento e cronograma ◆ MADS 26
5.2 Gestão de riscos ◆ MADS 27

5.3 Controlo de versões e trabalho em equipa (Git/GitHub) 27
6. Conclusão 28
7. Anexos 29

Índice de figuras e tabelas

Figura 1 – Diagrama de casos de uso
Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)
Figura 3 – Diagrama de classes (simplificado)
Figura 4 – Modelo de dados da base de dados SQLite (proposta)
Figura 5 – Máquina de estados do jogo
Figura 6 – Fluxograma do ciclo principal do jogo
Figura 7 – Fluxograma de decisão da inteligência artificial
Figura 8 – Captura de ecrã do StickMania durante uma partida

Tabela 1 – Descrição do caso de uso «Jogar partida»
Tabela 2 – Requisitos de hardware e software para correr o jogo
Tabela 3 – Parâmetros dos níveis de dificuldade da IA
Tabela 4 – Controlos do jogo
Tabela 5 – Plano de testes
Tabela 6 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)
Tabela 7 – Cronograma do projeto
Tabela 8 – Gestão de riscos

Lista de siglas e abreviaturas

2D – Duas dimensões

FPS – Frames (imagens) por segundo

GR – Guarda-redes

HUD – Heads-Up Display – informação mostrada por cima do jogo (marcador, mini-mapa)

IA – Inteligência Artificial

PAP – Prova de Aptidão Profissional

PNG – Portable Network Graphics – formato de imagem

SQL – Structured Query Language – linguagem de bases de dados

UML – Unified Modeling Language – linguagem de diagramas de software

1. Introdução

1.1 Enquadramento e contextualização do tema

O hóquei em patins é uma modalidade com grande tradição em Portugal. A seleção nacional é uma das mais vitoriosas do mundo e clubes como o FC Porto, o Benfica, o Sporting, o OC Barcelos ou a UD Oliveirense enchem pavilhões todas as épocas. Apesar disso, quase não existem videojogos atuais sobre a modalidade: há muitos jogos de futebol e de basquetebol, mas o hóquei em patins ficou esquecido.

O StickMania é um jogo de computador 2D de hóquei em patins, feito em Python com a biblioteca pygame-ce. O jogador controla uma equipa de cinco elementos (quatro jogadores de campo e um guarda-redes) contra uma equipa controlada pelo computador, num rinque visto de lado e de cima, como numa transmissão de televisão. O projeto foi desenvolvido no âmbito da Prova de Aptidão Profissional (PAP) do Curso Profissional de Técnico de Gestão e Programação de Sistemas Informáticos.

1.2 Motivação e pertinência do projeto

A ideia nasceu do gosto pela modalidade e da vontade de ver um jogo com as equipas portuguesas. Ao mesmo tempo, um jogo é um projeto muito completo para aprender programação: obriga a tratar de física (movimento, atrito, colisões e ressaltos), inteligência artificial, interfaces gráficas, animação, organização do código em módulos e testes.

O projeto é pertinente porque junta quase todas as matérias do curso num produto que qualquer pessoa consegue experimentar, e porque ajuda a divulgar uma modalidade com pouca presença nos videojogos.

1.3 Objetivos (geral e específicos)

Objetivo geral: desenvolver um jogo de hóquei em patins 2D jogável, estável e fluido, em que um jogador humano defronta uma equipa controlada pelo computador.

Objetivos específicos:

- Simular a física da bola e dos jogadores (aceleração, atrito do piso, ressaltos nas tabelas e nas balizas).

- Criar uma inteligência artificial com papéis táticos (avançado, médio e defesa) e três níveis de dificuldade.
- Implementar as regras principais: duas partes com tempo, golos, faltas e reposição da bola ao centro.
- Criar menus completos: menu principal, seleção de equipas, controlos, pausa e ecrã final.
- Mostrar as oito equipas com símbolos e equipamentos com as cores de cada clube.
- Desenhar o rinque com uma câmara em perspetiva, bancadas com público e balizas em 3D.
- Guardar torneios, partidas e classificação numa base de dados SQLite (fase seguinte do projeto).
- Manter o código organizado em módulos e com controlo de versões no GitHub.

1.4 Metodologia de trabalho (processo adotado; planeamento) ◆ MADS

Foi seguido um processo de desenvolvimento iterativo e incremental. Em vez de fazer tudo de uma vez, o jogo foi construído em pequenas versões, cada uma jogável, que foram sendo melhoradas:

- **Versão 1 –** rinque visto de cima, jogadores a mexer e bola com física simples.
- **Versão 2 –** física da bola corrigida, colisões, guarda-redes e golos.
- **Versão 3 –** interface do mockup (marcador, mini-mapa, cartão do jogador) e símbolos dos clubes.
- **Versão 4 –** mira de remate com o rato, bola colada ao stick, níveis de dificuldade e faltas.
- **Versão 5 –** animações de patinagem, balizas desenhadas no código e câmara em perspetiva com bancadas.

No fim de cada versão o jogo era testado e as falhas encontradas entravam na lista da versão seguinte. Todo o código está guardado no GitHub, o que permite ver a evolução e voltar atrás quando uma alteração não resulta (por exemplo, uma folha de sprites nova que acabou por ser revertida). O planeamento detalhado está no capítulo 5.

1.5 Estrutura do relatório

Depois desta introdução, o capítulo 2 faz a análise do sistema: o problema, os intervenientes, os requisitos e a modelação com diagramas (casos de uso, arquitetura, classes, estados e fluxogramas). O capítulo 3 descreve a implementação, os algoritmos principais e as dificuldades ultrapassadas. O capítulo 4 apresenta os testes realizados e

os resultados. O capítulo 5 trata da gestão do projeto (cronograma, riscos e controlo de versões). Por fim, o capítulo 6 apresenta a conclusão e o capítulo 7 os anexos.

2. Análise do sistema

2.1 Descrição do problema e da necessidade a resolver ◆ MADS

Quem gosta de hóquei em patins praticamente não tem jogos de computador da modalidade. Os poucos que existem são antigos, não têm as equipas portuguesas ou não reproduzem bem o jogo: a bola rápida, os ressaltos nas tabelas, o stick e o jogo atrás da baliza.

A necessidade a resolver é ter um jogo simples de instalar, que funcione num computador normal sem internet, com as equipas portuguesas, e que seja divertido tanto para quem conhece a modalidade como para quem nunca a viu. Para isso o jogo tem de ter uma física credível, adversários controlados pelo computador que joguem de forma inteligente e uma apresentação parecida com a televisão.

2.2 Intervenientes e contexto de utilização ◆ MADS

Intervenientes:

- **Jogador:** a pessoa que usa o jogo. Escolhe as equipas e a dificuldade e controla um jogador de cada vez.
- **Computador (IA):** controla a equipa adversária, os colegas do jogador e os dois guarda-redes.
- **Equipa de desenvolvimento:** Rafael da Hora Silveira e Vasco Andreia, responsáveis pela análise, programação, testes e documentação.
- **Orientadores:** Dr. Fernando Filipe e professora Cristina Paes, que acompanharam e validaram o trabalho.
- **Júri da PAP:** avalia o produto final e este relatório.

Contexto de utilização: o jogo corre num computador com Windows, numa janela de 1280×720 píxeis, e usa teclado e rato. Não precisa de internet nem de criar conta. Arranca com um duplo clique no ficheiro jogar.bat, que procura o Python e instala o pygame-ce se for preciso.

2.3 Levantamento e análise de requisitos ◆ MADS

Os requisitos dizem o que o jogo tem de fazer e em que condições. Foram tirados do anteprojeto do Stick Mania, a partir do problema, dos objetivos e da organização do jogo. Há requisitos funcionais, que descrevem o que o sistema faz, e requisitos não funcionais, ligados à segurança, à autonomia e à robustez.

2.3.1 Técnicas de levantamento utilizadas ◆ MADS

Para chegar a estes requisitos usámos quatro formas.
Leitura do anteprojeto. Lemos o documento e apontámos o que o jogo tem de ter: física, inteligência artificial, menus, torneio e base de dados.
Análise da modalidade. Usámos as regras básicas do hóquei em patins: duas equipas, guarda-redes, bola, stick, golos, tempo e classificação.
Comparação com outros jogos. Há muitos jogos de futebol e de basquetebol, mas quase nenhum de hóquei em patins atual. Isso mostrou o que faltava.
Revisão em grupo. Rafael Hora Silveira e Vasco Andreia reviram a lista com a professora Cristina Paes. Ainda não foram feitos inquéritos nem entrevistas.

2.3.2 Requisitos funcionais (o que o sistema deve fazer) ◆ MADS

Estes requisitos descrevem o que o jogo deve fazer.
01. O jogo deve abrir uma janela e continuar a correr, lendo o teclado.
02. Deve ter menu principal, escolha de equipas, pausa, definições e ecrã de resultado.
03. Deve deixar pausar e continuar sem perder o marcador nem o tempo.
04. Deve contar o tempo, separar as duas partes, detetar o golo e repor a bola no centro.
05. Deve mostrar o nome ou o emblema das duas equipas.
06. Deve mostrar jogadores de campo e guarda-redes, com stick e bola.
07. Deve mudar a animação do jogador: parado, patinar, conduzir, passar, rematar e chocar.
08. Deve mover o jogador ativo em várias direções, com aceleração e um limite de velocidade.
09. Deve deixar trocar o jogador que está a ser controlado.
10. Deve simular o atrito do rinque de madeira.
11. Deve calcular o movimento da bola, os ressaltos, a condução no stick, o passe e o remate.
12. Deve detetar colisões entre jogadores, bola, balizas e limites do rinque.
13. A equipa do computador deve perseguir a bola, defender, passar e rematar.
14. O guarda-redes deve mover-se na baliza para tentar defender.
15. Durante o jogo deve aparecer o marcador, o tempo, a parte e a pausa.
16. Deve haver um mini-mapa com os dez jogadores e a bola, e os atributos do jogador ativo.
17. No torneio deve atualizar vitórias, empates, derrotas e golos.
18. Deve guardar equipas, jogadores, partidas, estatísticas e classificação em SQLite.
19. Deve dar 3 pontos pela vitória, 1 pelo empate e 0 pela derrota, e ordenar a classificação.

2.3.3 Requisitos não funcionais (segurança, autonomia, robustez) ◆ MADS

Estes requisitos descrevem a segurança, a autonomia e a robustez do jogo.
01. As consultas à base de dados devem ser parametrizadas, para ninguém enviar comandos pela interface.
02. Os dados do torneio ficam só no computador. Não é preciso criar conta nem enviar dados para fora.
03. O jogo não guarda dados pessoais sensíveis. Guarda equipas, jogadores, partidas e estatísticas.
04. O jogo funciona sem internet e sem servidor.
05. Dá para continuar o torneio mais tarde, no mesmo computador.
06. O jogo não deve fechar sozinho ao pausar, ao marcar um golo ou ao mudar de parte.
07. A bola e os jogadores não devem ficar presos fora do rinque.
08. Se a gravação falhar, o que já estava guardado não deve ser apagado.
09. Não pode haver resultados impossíveis, como golos negativos.
10. O jogo deve andar de forma fluida, perto de 60 imagens por segundo.
11. O marcador, o tempo e o mini-mapa devem estar visíveis durante a partida.
12. O código fica dividido por partes, para cada um mudar a sua parte sem estragar o resto.

2.4 Modelação do sistema ◆ MADS

2.4.1 Cenários de utilização / casos de uso ◆ MADS

O diagrama de casos de uso mostra o que cada ator pode fazer no sistema. O Jogador escolhe as equipas e a dificuldade, vê os controlos, joga a partida, pausa e vê o resultado. Durante a partida pode mover o jogador, passar, rematar com mira e trocar de jogador. O Computador controla a equipa adversária e os guarda-redes. Os casos a amarelo e tracejado (torneio e classificação) estão previstos para a fase seguinte.



Figura 1 – Diagrama de casos de uso

Tabela 1 – Descrição do caso de uso «Jogar partida»

Campo	Descrição
Ator principal	Jogador (o Computador participa como ator secundário)
Pré-condição	As equipas e a dificuldade foram escolhidas no ecrã de seleção.
Fluxo principal	1. O sistema coloca os jogadores e a bola no centro. 2. O jogador move-se com WASD, passa com Z e remata com X (com mira). 3. A IA controla os restantes jogadores. 4. Quando a bola entra na baliza, o golo é somado e a bola volta ao centro. 5. No fim da 1.ª parte há intervalo; no fim da 2.ª parte aparece o resultado final.
Fluxos alternativos	ESC ou P abre a pausa (continuar, reiniciar, controlos, menu ou sair). Uma entrada irregular é assinalada como falta e a bola é reposta.
Pós-condição	O resultado final é mostrado; o jogador pode jogar de novo, mudar de equipas ou voltar ao menu.

2.4.2 Arquitetura do sistema (diagrama de blocos: sensores → controlo → atuadores) ◆ MADS

O modelo «sensores → controlo → atuadores» foi adaptado a um sistema de software: os «sensores» são o teclado e o rato, o «controlo» é o ciclo principal do jogo e os «atuadores» são o ecrã (e, mais tarde, o som). Por baixo ficam os dados que o jogo carrega ao arrancar. Cada bloco corresponde a um ou mais ficheiros Python, o que permite alterar uma parte sem estragar as outras.

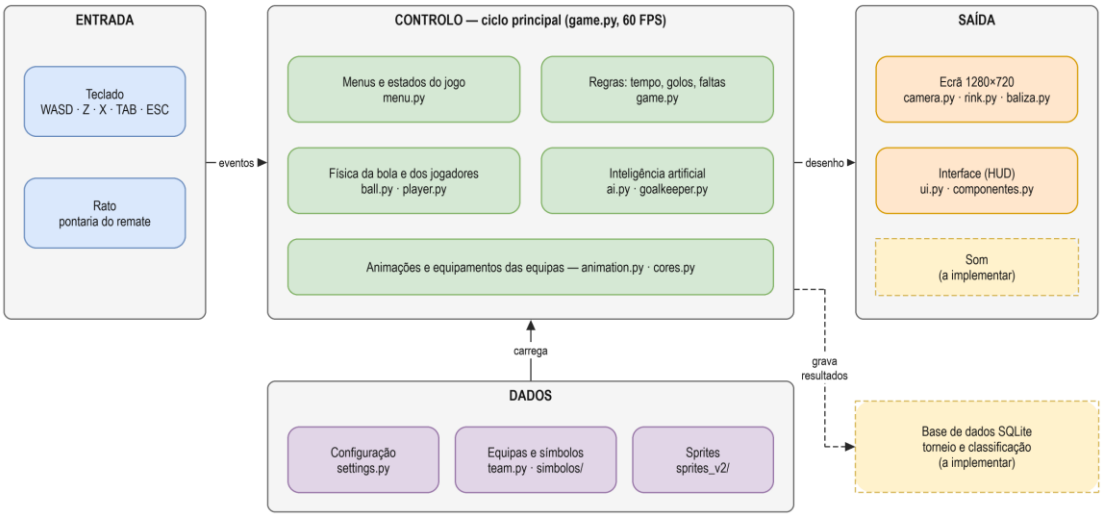


Figura 2 – Arquitetura do sistema em blocos (entrada → controlo → saída)

O diagrama de classes mostra as classes principais e como se ligam. A classe Game é o centro: tem o rinque, a bola, os dez jogadores de campo, os dois guarda-redes, os nove agentes de IA (todos os jogadores menos o humano), a interface e os menus. A Camera pertence ao Rink e é usada para desenhar tudo em perspetiva.

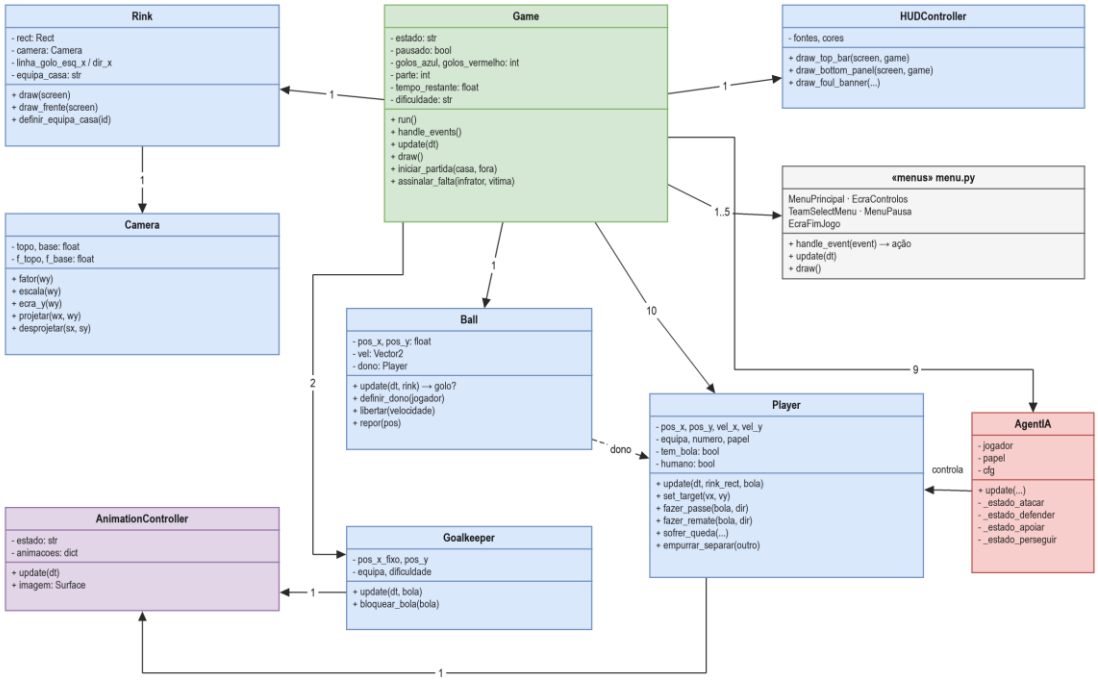


Figura 3 – Diagrama de classes (simplificado)

Para o modo torneio está prevista uma base de dados SQLite. O modelo de dados seguinte mostra as tabelas e as relações: uma equipa tem vários jogadores, um torneio tem várias partidas e uma linha de classificação por equipa, e cada partida guarda as estatísticas de cada jogador.

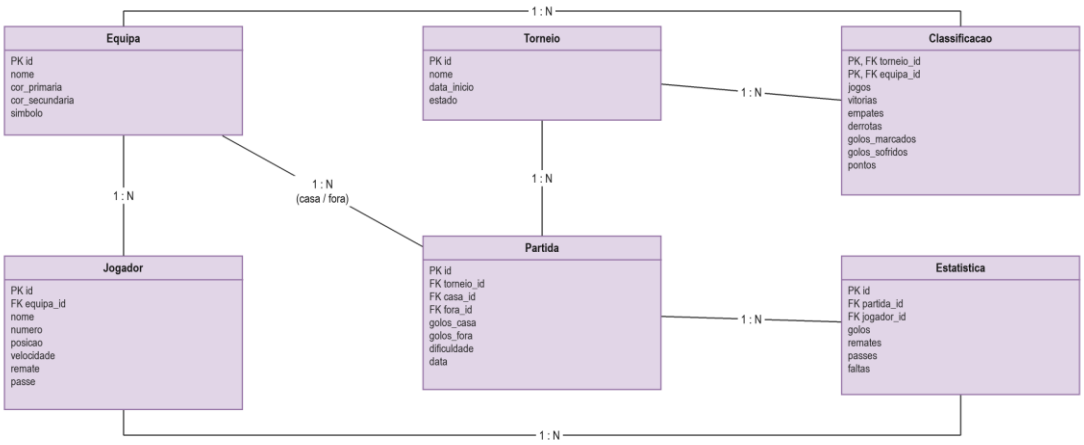


Figura 4 – Modelo de dados da base de dados SQLite (proposta)

2.4.3 Modelação comportamental (fluxogramas / máquina de estados) ◆ MADS

O jogo funciona como uma máquina de estados: em cada momento está num estado (menu, seleção, partida, pausa, etc.) e as ações do jogador fazem-no mudar de estado. No código isto é a variável estado da classe Game, junto com as variáveis pausado e fim_jogo.

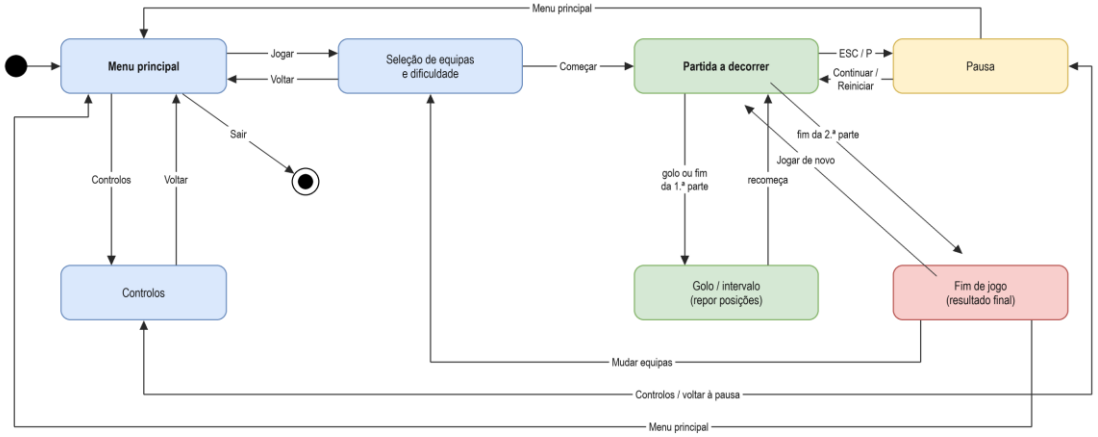


Figura 5 – Máquina de estados do jogo

O fluxograma seguinte mostra o ciclo principal, que se repete 60 vezes por segundo: ler o teclado e o rato, atualizar tudo e desenhar. A física e a lógica usam o tempo que passou desde o frame anterior (dt), para o jogo andar à mesma velocidade em computadores mais rápidos ou mais lentos.

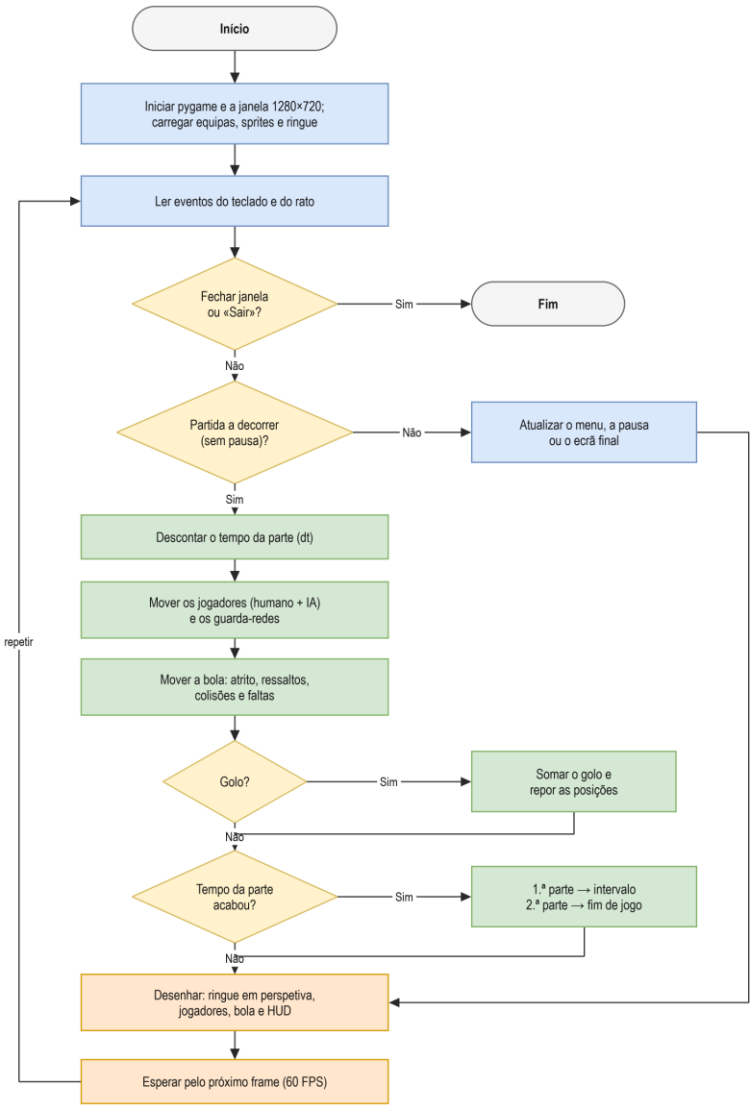


Figura 6 – Fluxograma do ciclo principal do jogo

Cada jogador da IA decide o que fazer em cada frame. Se tem a bola, ataca: contorna a baliza se estiver atrás dela, remata para o canto mais longe do guarda-redes se estiver perto e com a linha livre, passa a um colega desmarcado se estiver pressionado, ou conduz a bola fintando quem aparece à frente. Se não tem a bola e o adversário tem, os jogadores organizam-se numa fila: o mais perto do portador pressiona, o segundo faz a cobertura entre a bola e a baliza e os outros fecham os espaços. Quando é um colega a ter a bola, abrem espaço para receber. Com a bola solta, só vai buscá-la o jogador a quem o passe se destina ou o mais perto, e vai ao ponto onde a bola vai estar, não onde está.

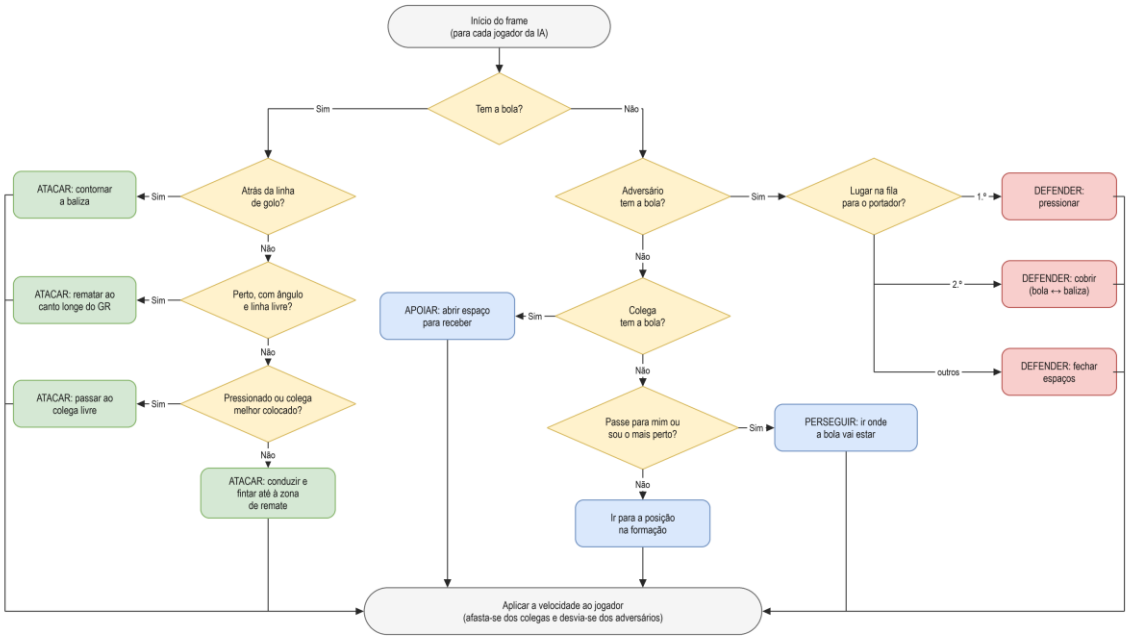


Figura 7 – Fluxograma de decisão da inteligência artificial

2.5 Projeto do hardware

O StickMania é um projeto só de software, por isso não há hardware a construir. Esta secção descreve o equipamento necessário para correr o jogo.

2.5.1 Componentes: sensores, atuadores e controlador

Não há sensores nem atuadores físicos. Os dispositivos usados são os de qualquer computador:

Tabela 2 – Requisitos de hardware e software para correr o jogo

Componente	Mínimo	Recomendado
Processador	Dual-core 2 GHz	Quad-core 2,5 GHz ou superior
Memória RAM	4 GB	8 GB
Ecrã	1280×720	1920×1080
Periféricos	Teclado	Teclado e rato
Sistema operativo	Windows 10	Windows 10 ou 11
Software	Python 3.10 + pygame-ce	Python 3.12 ou mais recente + pygame-ce

2.5.2 Esquema elétrico e ligações

Não se aplica: o projeto não tem circuitos elétricos.

2.5.3 Estrutura mecânica

Não se aplica: o projeto não tem estrutura mecânica.

2.6 Prototipagem (protótipo físico e/ou simulação) ◆ MADS

Em vez de um protótipo físico, foram feitos protótipos jogáveis do próprio jogo. O primeiro mostrava o rinque visto de cima, com jogadores e bola simples. As versões seguintes acrescentaram a física, a IA, a interface e, por fim, a câmara em perspetiva com bancadas, tabelas com altura e balizas em 3D.

Antes de programar a interface foi feito um mockup (desenho de como o ecrã devia ficar): marcador no topo, mini-mapa, cartão do jogador ativo e controlos em baixo. A figura seguinte mostra o estado atual do jogo.



Figura 8 – Captura de ecrã do StickMania durante uma partida

3. Implementação

3.1 Montagem do hardware

Não há hardware para montar. Em vez disso, foi preparado o ambiente de desenvolvimento:

- **Linguagem:** Python 3 (versão 3.10 ou mais recente).
- **Biblioteca gráfica:** pygame-ce 2.5, para a janela, o desenho, o teclado, o rato e o tempo.
- **Editor:** Visual Studio Code.
- **Controlo de versões:** Git e GitHub (repositório RafaelSilveira23/Stickmaniapap).
- **Diagramas:** draw.io (diagrams.net).
- **Arranque:** o ficheiro jogar.bat procura um Python compatível, instala o pygame-ce se faltar e abre o jogo.

3.2 Programação do controlo (lógica e algoritmos)

O código está dividido em módulos, um por responsabilidade (ver a arquitetura na secção 2.4.2). Seguem-se os algoritmos mais importantes.

Ciclo principal. game.py tem a classe Game, com o método run() que repete: handle_events() (ler o teclado e o rato), update(dt) (atualizar tudo) e draw() (desenhar). O pygame limita o ciclo a 60 FPS e dt é o tempo, em segundos, desde o frame anterior.

Física da bola. Em cada frame a bola anda vel × dt. O atrito do piso de madeira multiplica a velocidade por 0,991 quando a bola vai rápida e por 0,975 quando vai lenta (abaixo de 120 px/s), e abaixo de 8 px/s a bola para. Ao bater numa tabela, a componente da velocidade perpendicular à tabela troca de sinal e é multiplicada pelo coeficiente de ressalto 0,78. Um pequeno efeito lateral (spin) torna os ressaltos menos previsíveis. O passe tem força 400 px/s e o remate 650 px/s.

Movimento e colisões dos jogadores. O jogador não muda de velocidade de repente: acelera até à velocidade pretendida (200 px/s, ou 260 px/s com Shift) com uma aceleração de 2000 px/s². As colisões entre jogadores são detetadas pela distância entre os centros (menos de 38 px). Quando chocam, acontecem três coisas: são separados (cada um anda metade da sobreposição); a velocidade com que vinham um contra o outro é anulada, como num choque inelástico, para não ficarem a empurrar-se e a tremer; e voltam a ser mantidos dentro do rinque e fora das balizas.

Desarmes. Quando um defensor choca com quem tem a bola, há uma tentativa de desarme com probabilidade p = 0,42 + (defesa do defensor − 0,5 × velocidade do portador − 30) / 100, limitada entre 15 % e 85 % (o jogador humano tem +18 %). Se ganha, a bola salta para o lado dele. Quem acabou de ganhar a bola fica 0,6 s protegido e quem a perdeu fica 0,8 s sem poder desarmar, o que acabou com o «pingue-pongue» da bola a mudar de dono várias vezes por segundo. Só há falta em choques fortes ou cargas por trás.

Inteligência artificial. Cada jogador da IA tem um papel (avançado, médio ou defesa) e uma posição na formação que acompanha a bola. Em cada frame escolhe um de quatro comportamentos (atacar, defender, apoiar ou perseguir), como mostra o fluxograma da secção 2.4.3. O movimento usa três forças somadas: ir para o alvo (abrandando ao chegar, para não passar e voltar para trás), afastar-se dos colegas que estão a menos de 62 px,

e, com a bola, desviar-se dos adversários que estão à frente. Antes de passar ou rematar, a IA verifica se a linha está livre, calculando a distância de cada adversário ao segmento entre a bola e o alvo. Os passes são feitos para onde o colega vai estar (posição + velocidade × tempo da bola) e o colega que vai receber vai ao encontro da bola. Quando a posse muda, a IA demora um pouco a reagir (tempo de reação), como uma pessoa.

Tabela 3 – Parâmetros dos níveis de dificuldade da IA

Parâmetro	Fácil	Normal	Difícil
Velocidade normal (px/s)	140	175	215
Velocidade em sprint (px/s)	205	265	320
Raio para ir à bola (px)	135	190	265
Tempo entre remates (s)	2,2	1,4	0,75
Distância máxima de remate (px)	170	225	290
Erro de pontaria	≈ 12,5°	≈ 4,5°	≈ 1°
Tempo de reação da IA (s)	0,35	0,20	0,08
Guarda-redes: velocidade (px/s)	170	230	280
Guarda-redes: tempo de reação (s)	0,22	0,16	0,11
Guarda-redes: alcance (px)	14	16	18
Guarda-redes: erro de leitura (px)	± 28	± 22	± 15

A dificuldade só se aplica à equipa adversária; os colegas e o guarda-redes do jogador humano ficam sempre no nível normal.

Guarda-redes. O guarda-redes só se move na vertical, sobre a linha de golo. Não é perfeito: guarda as últimas posições da bola e reage ao que viu há alguns décimos de segundo (tempo de reação). Quando há um remate, calcula onde a bola vai cruzar a linha de golo, y = y_bola + v_y × (x_GR − x_bola) / v_x, e vai para esse ponto, mas com um pequeno erro de leitura sorteado em cada remate. Se a bola passa ao seu alcance, bloqueia-a e afasta-a. Estes valores foram afinados com simulações até haver um número de golos realista.

Câmara em perspetiva. A física continua num campo plano visto de cima. A câmara só muda o desenho: o campo passa a ser um trapézio, mais estreito ao fundo. Para uma posição t (0 = fundo, 1 = frente), a escala horizontal é f(t) = f_topo + (f_base − f_topo)·t e a posição vertical no ecrã é proporcional ao integral G(t) = f_topo·t + (f_base − f_topo)·t²/2. Para saber onde o rato aponta no campo é preciso fazer a conta ao contrário, o que dá uma equação do 2.º grau, resolvida com a fórmula resolvente. Os jogadores do fundo são desenhados mais pequenos e antes dos da frente (ordenados pela posição y), para ficarem por trás.

Equipamentos das equipas. Os sprites dos jogadores são desenhados com um equipamento azul. Ao carregar a partida, os píxeis com tom (hue) entre 180° e 260° são pintados com as cores do clube, mantendo as sombras. As imagens recoloridas ficam guardadas em cache para não repetir o trabalho.

Animações. Cada jogador tem um controlador de animações com estados (parado, correr, com bola, passe, remate, queda). A velocidade da animação acompanha a velocidade do

jogador, e há uma inclinação para a frente ao patinar e um pequeno efeito ao mudar de direção.

3.3 Integração hardware – software

Neste projeto, a integração é entre o jogo e os periféricos do computador. A entrada é lida com os eventos do pygame (teclas premidas e cliques) e com o estado do teclado em cada frame (para o movimento contínuo). A saída é desenhada numa superfície de 1280×720 que é mostrada no ecrã 60 vezes por segundo.

Tabela 4 – Controlos do jogo

Tecla / botão	Ação
W A S D	Mover o jogador ativo
Shift	Sprint (patinar mais depressa)
Z	Passar a bola
X	Abrir a mira de remate; o rato escolhe a direção e o clique esquerdo remata
TAB / C / Espaço	Trocar para o jogador mais perto da bola
ESC / P	Pausa (ESC com a mira aberta cancela o remate)
T / Y	Trocar a equipa da casa / de fora
F4	Mudar a dificuldade durante o jogo
F3	Mostrar informação de depuração

3.4 Decisões técnicas e dificuldades ultrapassadas

Perspetiva sem mudar a física. A primeira ideia foi transformar toda a física para 3D. Em vez disso, a física ficou num campo plano e só o desenho passou pela câmara. Assim a IA e as colisões não mudaram e não apareceram erros novos.

Balizas desenhadas no código. Os PNG antigos das balizas estavam desenhados de lado e não combinavam com o rinque. As balizas passaram a ser desenhadas no código a partir de pontos 3D projetados pela câmara, em duas camadas: a rede de trás (desenhada antes dos jogadores) e os postes da frente (desenhados depois). Assim a bola e o guarda-redes parecem estar dentro da baliza.

Desempenho. Desenhar o rinque em perspetiva linha a linha é lento. Por isso, o rinque, as tabelas e as bancadas são desenhados uma só vez, quando a partida começa, e depois só se copia essa imagem em cada frame. O jogo arranca em menos de um segundo e mantém os 60 FPS.

Pontaria com o rato. Com a câmara, a posição do rato no ecrã já não é a posição no campo. Foi preciso escrever a função inversa da câmara (desprojetar), com a equação do 2.º grau descrita na secção 3.2.

Sprites. Foi criada uma folha de sprites nova com mais animações, mas o resultado visual ficou pior do que o original e a alteração foi revertida com o Git. A melhoria dos sprites fica para uma fase seguinte.

Bola presa e jogadores colados. Ao início a bola atravessava as tabelas quando ia muito rápida e os jogadores ficavam presos uns nos outros. Foi corrigido limitando a velocidade, verificando a posição depois de cada movimento e separando os jogadores que se sobrepõem.

Colisões e IA com comportamentos estranhos. Mesmo depois disso, os jogadores ainda «tremiam» quando iam uns contra os outros, a bola mudava de dono várias vezes por segundo e a IA tinha comportamentos estranhos: os avançados não voltavam para defender, ninguém ia a uma bola parada entre dois colegas, e quase não havia golos porque o guarda-redes tapava a baliza toda e reagia instantaneamente. Para resolver isto sem «adivinhar», foi criado um script que joga partidas IA contra IA sem janela e conta os problemas (Tabela 6). Cada correção foi medida antes e depois. Também se descobriu que, depois de qualquer golo, a bola ia sempre para a equipa do jogador (agora vai para quem sofreu o golo) e que a dificuldade «Difícil» também melhorava os colegas do jogador.

4. Testes e validação

4.1 Plano de testes

Os testes foram de dois tipos: testes manuais (jogar e verificar cada funcionalidade) e testes automáticos, com scripts que arrancam o jogo sem janela (SDL_VIDEODRIVER=dummy), simulam teclas e verificam o resultado. A tabela resume o plano de testes.

Tabela 5 – Plano de testes

N.º	Teste	Resultado esperado	Resultado
T01	Arrancar o jogo com jogar.bat	Abre o menu principal	Passou
T02	Escolher equipas e dificuldade e começar	A partida começa com as equipas escolhidas	Passou
T03	Mover o jogador com WASD e Shift	Move-se nas 8 direções e acelera com Shift	Passou
T04	Passar (Z) e rematar com mira (X + rato)	A bola sai na direção escolhida	Passou
T05	Bola contra as tabelas a alta velocidade	Ressalta e nunca sai do rinque	Passou
T06	Bola a entrar na baliza	Golo somado à equipa certa e bola ao centro	Passou
T07	Fim da 1.ª e da 2.ª parte	Intervalo e depois ecrã de resultado final	Passou
T08	Pausar e continuar	Marcador e tempo mantêm-se	Passou

N.º	Teste	Resultado esperado	Resultado
T09	Trocar equipas durante o jogo (T / Y)	Símbolos, cores e equipamentos mudam	Passou
T10	Rato em perspetiva (projetar e desprojetar)	Erro de 0 px na conversão ida e volta	Passou
T11	Desempenho	60 FPS estáveis durante a partida	Passou
T12	Jogadores a chocar uns com os outros	Separam-se sem tremer nem ficar sobrepostos	Passou
T13	Disputa de bola entre dois jogadores	A bola não muda de dono várias vezes seguidas	Passou
T14	IA defende quando o adversário tem a bola	O mais perto pressiona e outro faz cobertura	Passou
T15	Golo marcado	Recomeça com a bola a equipa que sofreu o golo	Passou
T16	Mudar a dificuldade	Só muda a equipa adversária	Passou
T17	Gravar o torneio em SQLite	Dados guardados e lidos corretamente	Por fazer

4.2 Ensaios, calibração e medições

Foram feitas medições com o jogo a correr sem limite de FPS, em simulação automática:

- **Tempo de arranque:** cerca de 0,75 s até a partida estar pronta (inclui desenhar o rinque em perspetiva).
- **Velocidade de desenho:** entre 250 e 290 frames por segundo sem limite, muito acima dos 60 FPS necessários.
- **Precisão da câmara:** converter um ponto do campo para o ecrã e de volta dá erro de 0,0 píxeis.
- **Golos:** em simulações com remates automáticos, todos os golos foram detetados e atribuídos à equipa certa.

Também foram afinados vários valores por tentativa e erro: o coeficiente de ressalto passou de 0,72 para 0,78 (a bola parecia «morta»), a velocidade de sprint passou de 330 para 260 px/s (era demasiado rápida), o tamanho da bola passou de 14 para 11 px e o atrito foi dividido em dois valores (bola rápida e bola lenta) para a bola parar de forma mais natural.

Para a jogabilidade foi criado um script de medição que põe as duas equipas controladas pela IA a jogar sem janela, 8 jogos de 3 minutos, sempre com as mesmas sementes aleatórias (para a comparação ser justa), e conta os problemas. A tabela compara a versão antiga com a versão corrigida.

Tabela 6 – Medição da jogabilidade antes e depois das correções (média por jogo de 3 min, IA contra IA)

Medida	Antes	Depois
Golos	0,4	2,0
Remates	20	50

Medida	Antes	Depois
Bola roubada e devolvida em menos de 0,6 s («pingue-pongue»)	101	16
Passes (muitos eram intercetados)	378	80
Faltas apitadas	15,1	0,5
Frames com jogadores a «tremer» (inverter o sentido)	59	31
Momentos com jogadores sobrepostos	4,8	0,6
Momentos com jogadores fora do rinque	0,4	0

4.3 Resultados obtidos e ajustes

Os testes T01 a T16 passaram. O teste T17 ainda não foi feito porque o torneio com base de dados é a próxima fase do projeto.

Os testes levaram a vários ajustes: o triângulo de seleção e a etiqueta «GR» ficavam a flutuar acima dos jogadores depois da câmara em perspetiva e passaram a usar o retângulo visível do sprite; o stick ficava demasiado claro e foi escurecido; as tabelas e as balizas foram redesenhadas para ficarem iguais à imagem de referência; e a IA foi afinada para haver menos quedas.

Na afinação do guarda-redes testaram-se várias combinações de alcance, erro de leitura e tempo de reação. Com o alcance antigo (34 px) o guarda-redes tapava a baliza toda e quase não havia golos; com antecipação perfeita da trajetória também não. Os valores finais dão cerca de 4 % de remates convertidos em golo entre duas equipas da IA, o que corresponde a 6 ou 7 golos num jogo de 2 × 5 minutos.

5. Gestão do projeto

5.1 Planeamento e cronograma ◆ MADS

A tabela seguinte mostra o cronograma do projeto, desde o anteprojeto até à defesa.

Tabela 7 – Cronograma do projeto

Período	Tarefa	Estado
set.–out. 2026	Anteprojeto, levantamento de requisitos, escolha das tecnologias	Concluído
out. 2026	Protótipo: rinque, jogadores, bola e física básica	Concluído
out. 2026	Interface do mockup, símbolos dos clubes e seleção de equipas	Concluído

Período	Tarefa	Estado
out. 2026	Mira de remate, IA com dificuldades, faltas	Concluído
out. 2026	Animações, balizas, câmara em perspetiva e bancadas	Concluído
out. 2026	Correção das colisões, nova IA tática e guarda-redes com reação	Concluído
nov.–dez. 2026	Som, estatísticas da partida, modo de 2 jogadores	Planeado
jan.–mar. 2027	Modo torneio com base de dados SQLite e classificação	Planeado
mar.–abr. 2027	Testes com colegas, correção de erros, novos sprites	Planeado
abr.–mai. 2027	Relatório final, manual de utilizador e apresentação	Em curso
jun. 2027	Defesa da PAP	Planeado

5.2 Gestão de riscos ◆ MADS

Tabela 8 – Gestão de riscos

Risco	Prob.	Impacto	Medida
Perder código (avaria, ficheiro apagado)	Média	Alto	Tudo no GitHub; commits frequentes
Uma alteração estragar o que funcionava	Alta	Médio	Testar antes de cada commit; reverter com o Git
Atrasos no cronograma	Média	Alto	Versões pequenas e jogáveis; prioridades definidas
Jogo lento em computadores fracos	Baixa	Médio	Pré-desenhar o rinque; medir os FPS
Erros de física (bola fora do rinque)	Média	Alto	Testes automáticos de colisões e golos
Uso dos símbolos dos clubes	Baixa	Médio	Uso apenas educativo, sem fins comerciais
Ausência de um elemento do grupo	Baixa	Médio	Código modular e documentado; repositório partilhado

5.3 Controlo de versões e trabalho em equipa (Git/GitHub)

O projeto usa o Git para o controlo de versões, com o repositório no GitHub (github.com/RafaelSilveira23/Stickmaniapap). Cada conjunto de alterações é guardado num commit com uma mensagem a explicar o que mudou, por exemplo «feat: níveis de dificuldade da IA, redução de quedas e sistema completo de faltas». Assim é possível ver o histórico, comparar versões e voltar atrás, como aconteceu com a folha de sprites que foi revertida.

O código está dividido em módulos (física, IA, interface, câmara, rinque), o que permite a cada elemento do grupo trabalhar numa parte sem estragar as outras. Os diagramas também estão no repositório (pasta docs/diagramas), em formato draw.io, para poderem ser editados.

6. Conclusão

O StickMania cumpre o objetivo principal: é um jogo de hóquei em patins jogável, fluido e com as equipas portuguesas. Tem física da bola com atrito e ressaltos, uma IA com papéis táticos e três níveis de dificuldade, guarda-redes, faltas, duas partes, menus completos e uma apresentação parecida com a televisão, com câmara em perspetiva, bancadas e balizas em 3D.

O projeto permitiu aplicar e aprofundar muitas matérias do curso: programação orientada a objetos, organização em módulos, matemática aplicada (vetores, equações do 2.º grau, perspetiva), algoritmos de decisão, testes e controlo de versões. As maiores dificuldades foram a física das colisões e a câmara em perspetiva, resolvidas separando a lógica do jogo da forma como é desenhado.

Como trabalho futuro estão previstos: o modo torneio com base de dados SQLite e classificação, sons (apito, bola, público), estatísticas no fim do jogo, um modo de dois jogadores no mesmo teclado e sprites novos com mais animações.

7. Anexos

Anexo A – Ficheiros dos diagramas. Todos os diagramas deste relatório estão na pasta docs/diagramas do projeto, em formato .drawio (abrem e editam-se em app.diagrams.net) e em imagem .png:

- 01_casos_de_uso.drawio – diagrama de casos de uso
- 02_arquitetura.drawio – arquitetura em blocos
- 03_estados_jogo.drawio – máquina de estados do jogo
- 04_fluxograma_ia.drawio – fluxograma da IA
- 05_fluxograma_ciclo.drawio – fluxograma do ciclo principal
- 06_classes.drawio – diagrama de classes
- 07_modelo_dados.drawio – modelo de dados SQLite

Anexo B – Manual de utilizador (resumo). 1) Instalar o Python 3.10 ou mais recente. 2) Fazer duplo clique em jogar.bat (instala o pygame-ce na primeira vez). 3) No menu, escolher «Jogar», as equipas e a dificuldade. 4) Jogar com os controlos da Tabela 4.

Anexo C – Código-fonte: github.com/RafaelSilveira23/Stickmaniapap